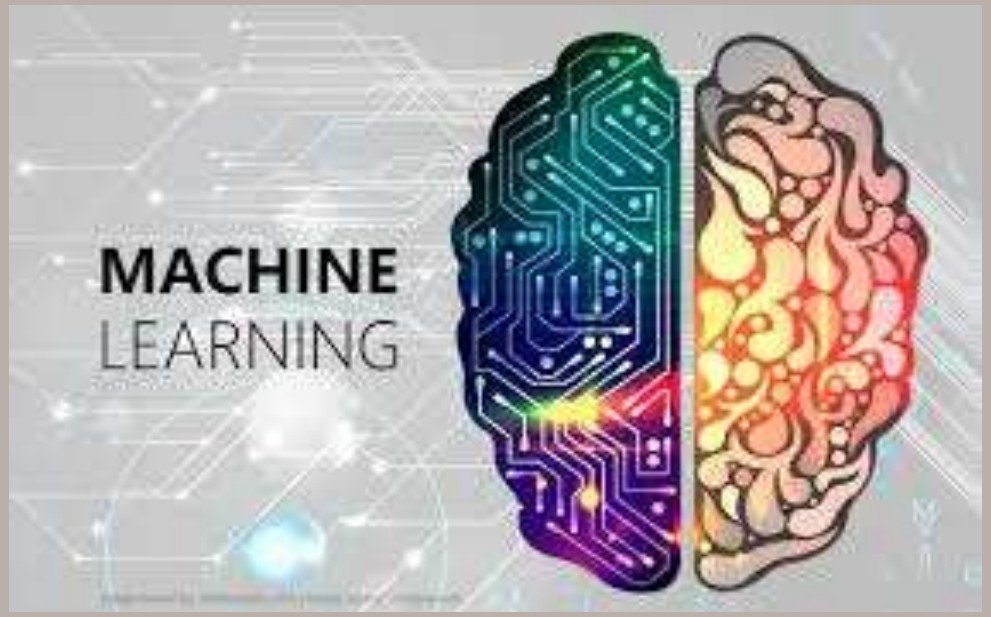
CSE 4137

Machine Learning
Module-05
KNN Classification



Md. Shahid Uz Zaman
Adjunct Faculty
Dept of CSE, BAUET
szaman22.ruet@gmail.com

Topics: KNN

Week-05

- Introduction
- Implementation
- Python Example

KNN (K-Nearest Neighbor)

Week-06, c-3

What is KNN?

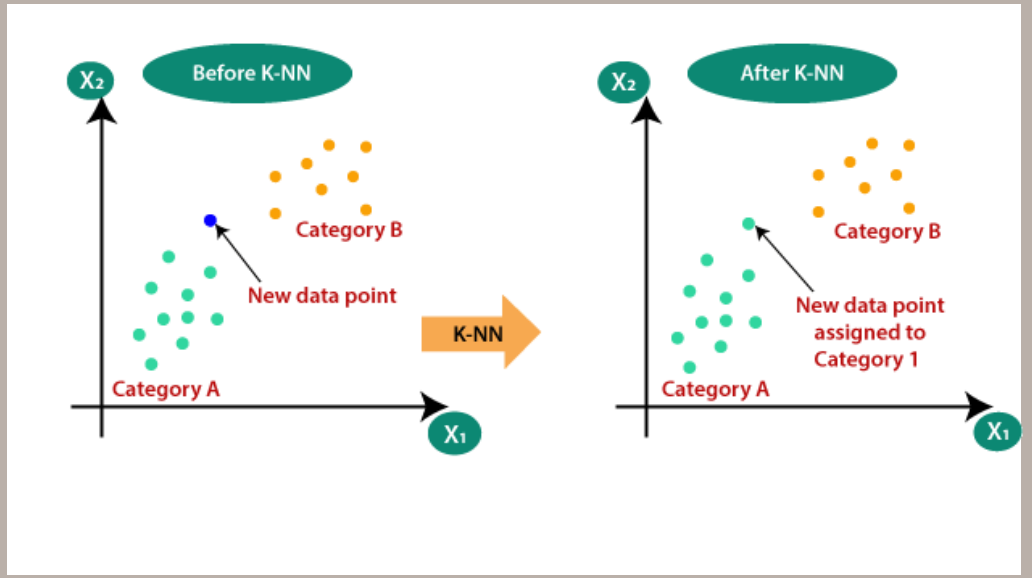
KNN is a simple, supervised machine learning (ML) algorithm that can be used for classification or regression tasks - and is also frequently used in missing value imputation. It is based on the idea that the observations closest to a given data point are the most "similar" observations in a data set, and we can therefore classify unforeseen points based on the values of the closest existing points. By choosing *K*, the user can select the number of nearby observations to use in the algorithm.

How does KNN works?

K is the number of nearest neighbors to use. For classification, a majority vote is used to determined which class a new observation should fall into. Larger values of *K* are often more robust to outliers and produce more stable decision boundaries than very small values (*K=3* would be better than *K=1*, which might produce undesirable results.

Example: KNN

Suppose there are two categories, i.e., Category A and Category B, and we have a new data point x1, so this data point will lie in which of these categories. To solve this type of problem, we need a K-NN algorithm. With the help of K-NN, we can easily identify the category or class of a particular dataset. Consider the below diagram:



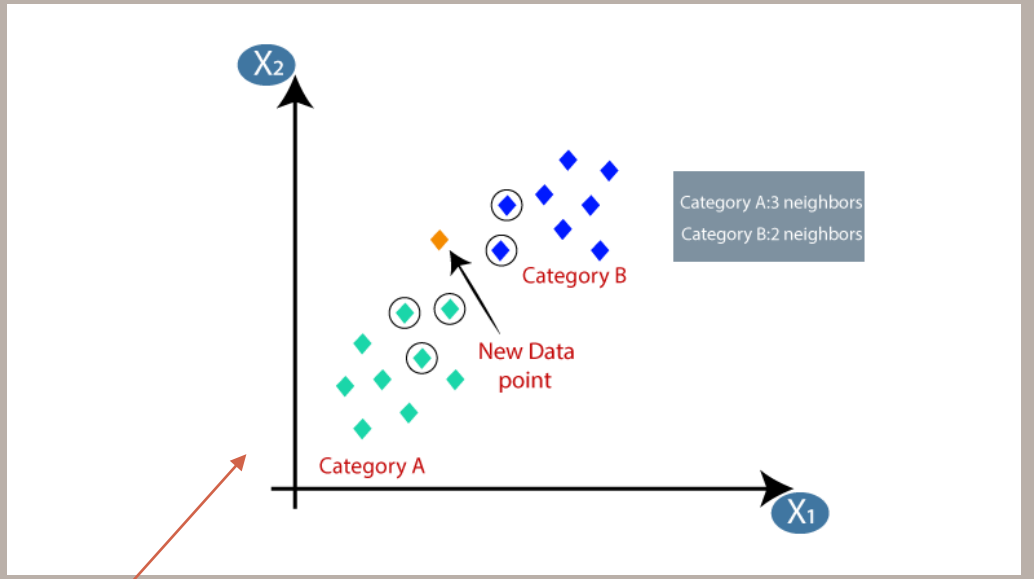
KNN (K-Nearest Neighbor)

Week-06, c-3

Steps of KNN Algorithm

The K-NN working can be explained on the basis of the below algorithm:

- **Step-1:** Select the number K of the neighbors
- **Step-2:** Calculate the Euclidean distance of **K number of neighbors**
- **Step-3:** Take the K nearest neighbors as per the calculated Euclidean distance.
- **Step-4:** Among these k neighbors, count the number of the data points in each category.
- **Step-5:** Assign the new data points to that category for which the number of the neighbor is maximum.
- **Step-6:** Our model is ready.



•By calculating the Euclidean distance we got the nearest neighbors, as three nearest neighbors in category A and two nearest neighbors in category B. Consider the below image:

•As we can see the 3 nearest neighbors are from category A, hence this new data point must belong to category A.

Advantages :KNN Algorithm

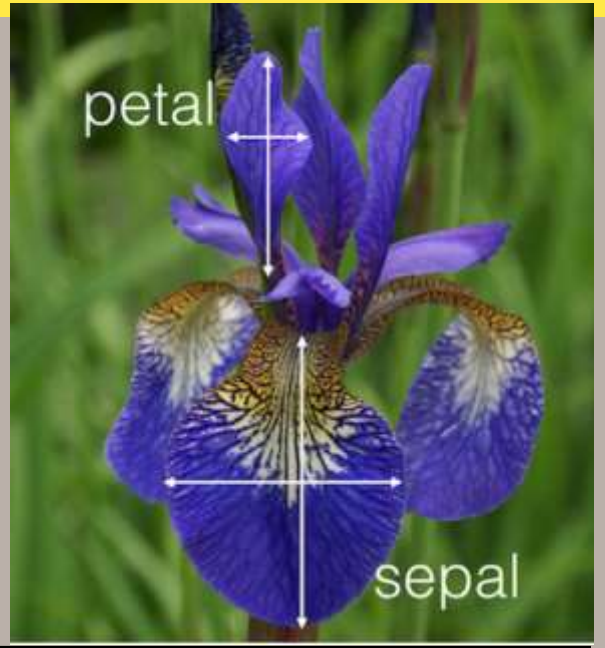
- It is simple to implement.
- It is robust to the noisy training data
- It can be more effective if the training data is large.

KNN (K-Nearest Neighbor)

Week-06, c-3

Example: Iris Dataset

The **Iris dataset** is one of the most well-known and commonly used datasets in the field of **machine learning** and statistics. In this article, we will explore the **Iris dataset in deep** and learn about its uses and applications.
The Iris dataset consists of 150 samples of iris flowers from three different species: Setosa, Versicolor, and Virginica. Each sample includes four features: sepal length, sepal width, petal length, and petal width. It was introduced by the British biologist and statistician Ronald Fisher in 1936 as an example of discriminant analysis.



This dataset is particularly popular due to its simplicity and the clear separation of the different species based on the features provided. The four features are all measured in centimeters.

- **Sepal Length:** The length of the iris flower's sepals (the green leaf-like structures that encase the flower bud).
- **Sepal Width:** The width of the iris flower's sepals.
- **Petal Length:** The length of the iris flower's petals (the colored structures of the flower).
- **Petal Width:** The width of the iris flower's petals.



KNN (K-Nearest Neighbor)

Week-06, c-3

Python: Iris Dataset

```
import pandas as pd
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt
%matplotlib inline
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sn
iris = load_iris()
```

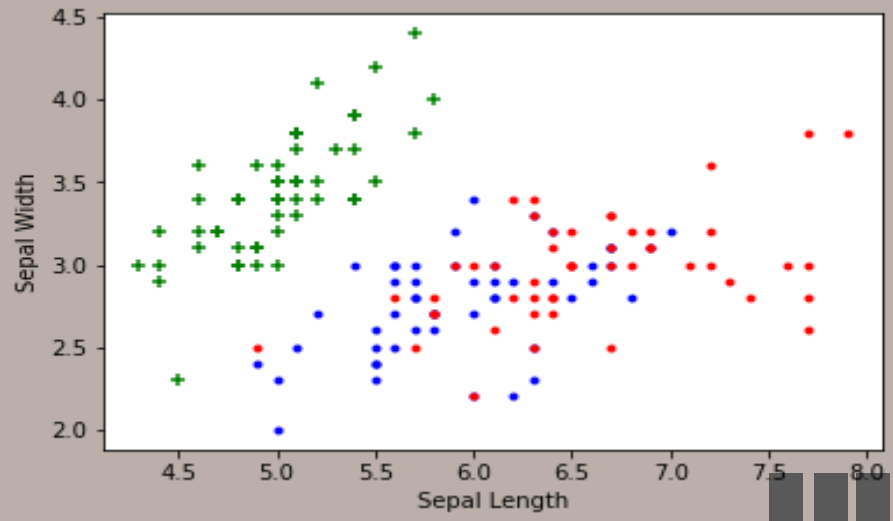
```
iris.feature_names
iris.target_names
```

```
df = pd.DataFrame(iris.data,columns=iris.feature_names)
df.head
```

```
df['target'] = iris.target
df.head()
```

```
df0 = df[:50]
df1 = df[50:100]
df2 = df[100:]
```

```
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
plt.scatter(df0['sepal length (cm)'], df0['sepal width (cm)'],color="green",marker='+')
plt.scatter(df1['sepal length (cm)'], df1['sepal width (cm)'],color="blue",marker='.')
plt.scatter(df2['sepal length (cm)'], df2['sepal width (cm)'],color="red",marker='.')
```

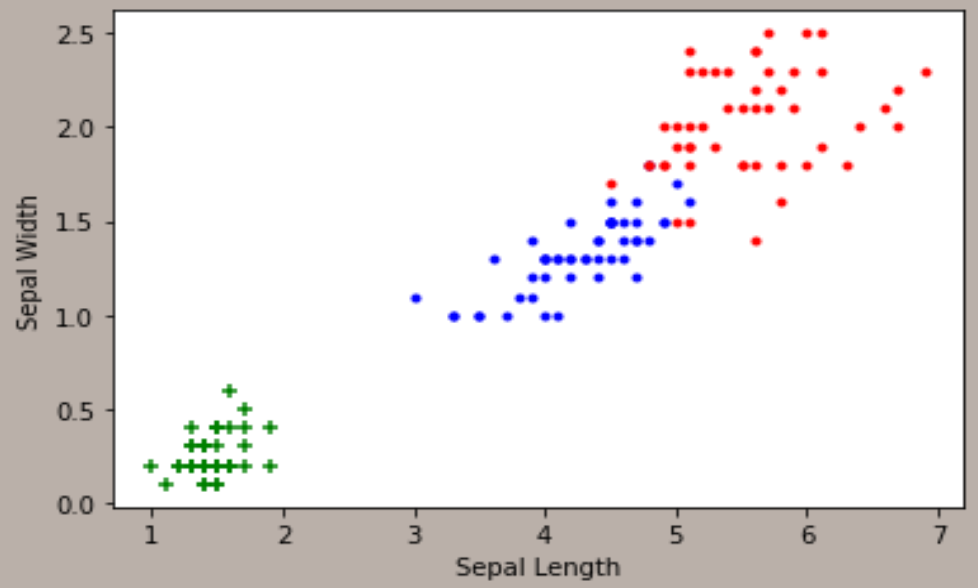


KNN (K-Nearest Neighbor)

Week-06, c-3

Python: Iris Dataset

```
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
plt.scatter(df0['petal length (cm)'], df0['petal width (cm)'],color="green",marker='+')
plt.scatter(df1['petal length (cm)'], df1['petal width (cm)'],color="blue",marker='.')
plt.scatter(df2['petal length (cm)'], df2['petal width (cm)'],color="red",marker='.')
```




```
X = df.drop(['target'], axis='columns')
y = df.target
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=1)
```

```
knn = KNeighborsClassifier(n_neighbors=10)
knn.fit(X_train, y_train)
```

```
knn.score(X_test, y_test)
```

```
knn.predict([[4.8,3.0,1.5,0.3]])
```

```
y_pred = knn.predict(X_test)
cm = confusion_matrix(y_test, y_pred)
cm
```

```
array([[11,  0,  0],
       [ 0, 12,  1],
       [ 0,  0,  6]], dtype=int64)
```

```
array([[11,  0,  0],
       [ 0, 12,  1],
       [ 0,  0,  6]], dtype=int64)
```

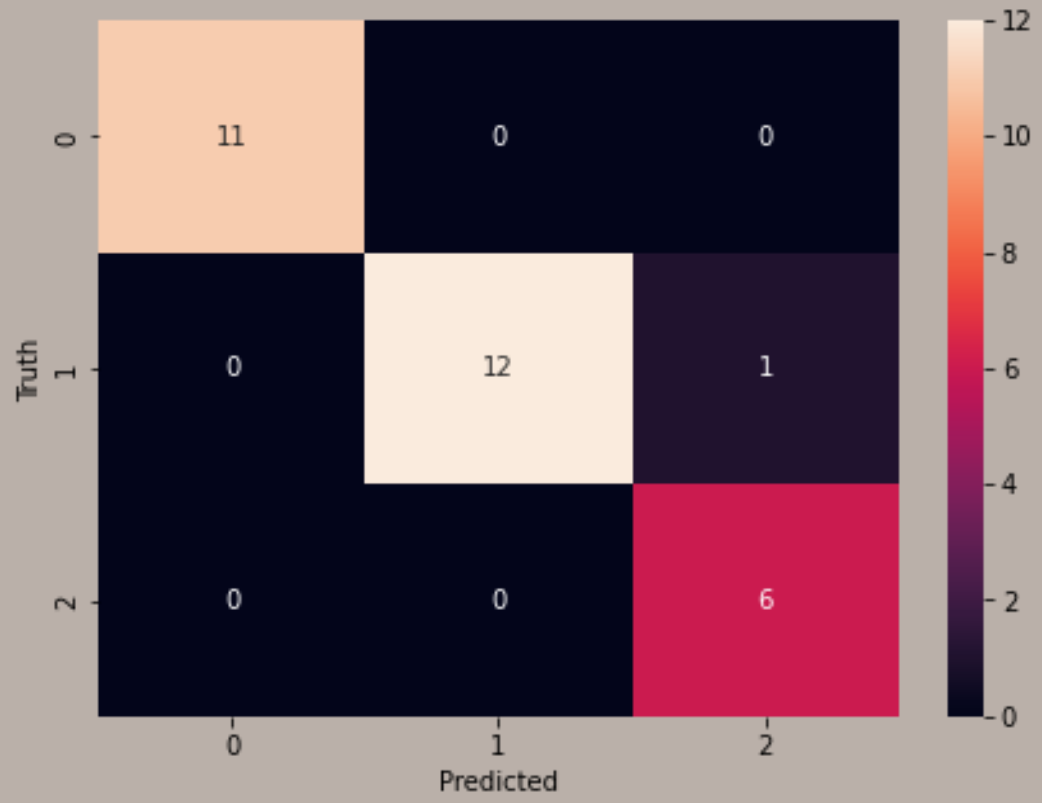
```
(classification_report(y_test, y_pred))
```

KNN (K-Nearest Neighbor)

Week-06, c-3

Python: Iris Dataset

```
plt.figure(figsize=(7,5))
sn.heatmap(cm, annot=True)
plt.xlabel('Predicted')
plt.ylabel('Truth')
```




```
print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	11
1	1.00	0.92	0.96	13
2	0.86	1.00	0.92	6
accuracy			0.97	30
macro avg	0.95	0.97	0.96	30
weighted avg	0.97	0.97	0.97	30

End of Chapter